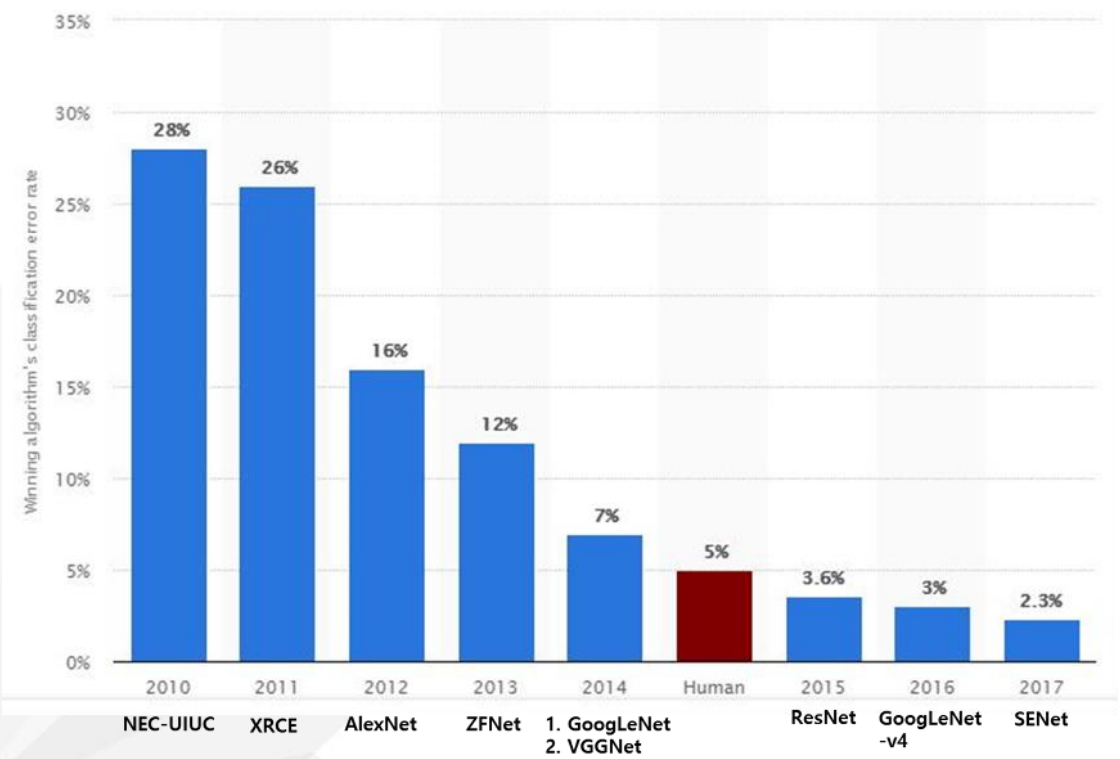
CNN (2)

국립금오공과대학교
시종욱

목차

- AlexNet
- VGGNet
- GoogleNet
- ResNet
- U-Net

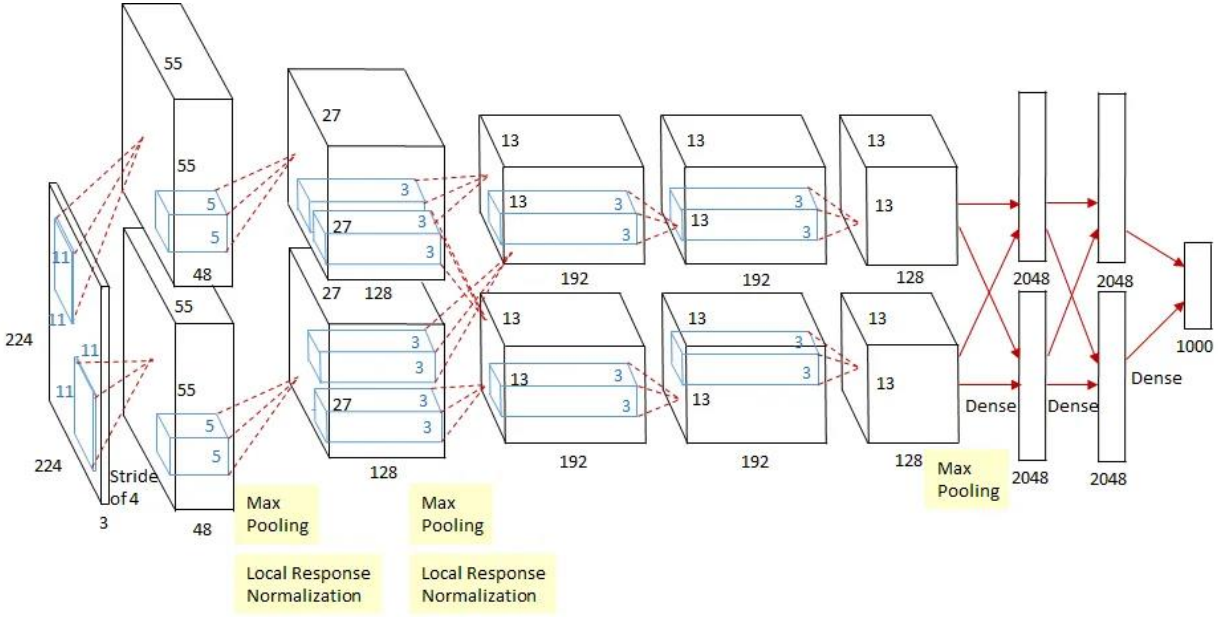
다양한 고전 모델



https://cvml.tistory.com/1

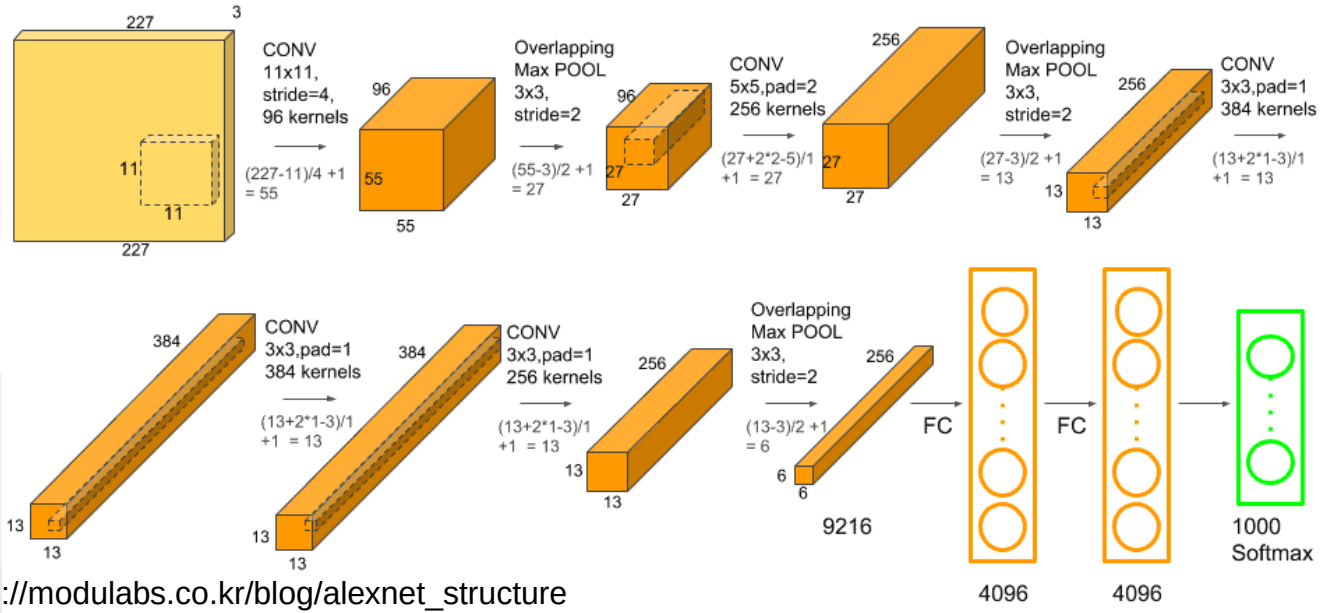
AlexNet

- AlexNet (2012)
  - CNN 부흥에 아주 큰 역할을 한 네트워크
  - ILSVRC(2012) 대회에서 우승을 차지한 CNN 구조



AlexNet

- AlexNet (2012)
  - ➢ 224×224×3 크기가 언급되어있지만, 이후 수정되어 1 계층에서 패딩을 통해 227x227x3 사이즈로 변경



https://modulabs.co.kr/blog/alexnet_structure

AlexNet

- AlexNet (2012)
  - ➢ 합성곱 계층 1
    - ✓ 11x11x3 크기의 커널 48개를 2개 그룹 사용하여, 55x55x(48x2) 특징맵 출력 (stride=4, padding=0)
    - ✓ 3x3 중첩 최대풀링 Overlapping Max Pooling 을 적용하여, 27x27x(48x2) 특징맵 출력 (stride=2)
    - ✓ 지역 응답 정규화 Local Response Normalization 를 적용하고, 27x27x(48x2) 특징맵을 출력
  - ➢ 합성곱 계층 2
    - ✓ 5x5x48 크기의 커널 128개를 2개 그룹 사용하여, 27x27x(128x2) 특징맵 출력 (stride=1, padding=2)
    - ✓ 3x3 중첩 최대풀링 적용하여, 13x13x128x2 특징맵 출력 (stride=2)
    - ✓ 지역 응답 정규화를 적용하고, 13x13x128x2 특징맵 출력

AlexNet

- AlexNet (2012)
  - ➢ 합성곱 계층 3
    - ✓ 3x3x256 크기의 커널 192개를 2개 그룹 사용하여, 13x13x192x2 특징맵 출력 (stride=1, padding=1)
  - ➢ 합성곱 계층 4
    - ✓ 3x3x192 크기의 커널 192개를 2개 그룹 사용하여, 13x13x192x2 특징맵 출력 (stride=1, padding=1)
  - ➢ 합성곱 계층 5
    - ✓ 3x3x192 크기의 커널 128개를 2개 그룹 사용하여, 13x13x128x2 특징맵 출력 (stride=1, padding=1)
    - ✓ 3x3 중첩 최대풀링(stride=2)을 적용하여, 6x6x128x2 특징맵 출력
  - ➢ 완전연결 계층 6-8
    - ✓ 5계층 합성곱 후 6계층, 7계층에서는 각각 4096개의 노드
    - ✓ 마지막 8계층에는 1000개의 노드가 있어 1000개 클래스 구분을 의미하는 확률값을 출력

AlexNet

- AlexNet (2012)
  - ➢ 중첩 최대 풀링(Overlapping Max Pooling)
    - ✓ 일반적인 최대 풀링은 커널 크기와 스트라이드 크기가 동일
    - ✓ 하지만 AlexNet에서는 3x3 크기 커널을 2픽셀 단위로 움직이며 pooling을 수행하는 중첩 최대 풀링을 사용
    - ✓ 중첩 pooling이 더 세밀한 특징을 추출할 수 있었음
  - ➢ 지역 응답 정규화(Local Response Normalization)
    - ✓ AlexNet에서는 합성곱 계층 후에 지역 응답 정규화 계층을 추가
    - ✓ 이 계층은 입력 매핑(feature map) 각각의 뉴런에 대해 주변 뉴런들의 활성화 정도에 의해 정규화를 수행
    - ✓ 학습 수렴 속도를 높여주었음 : 현재 대신 배치 정규화(Batch Normalization)가 주로 사용

VGG

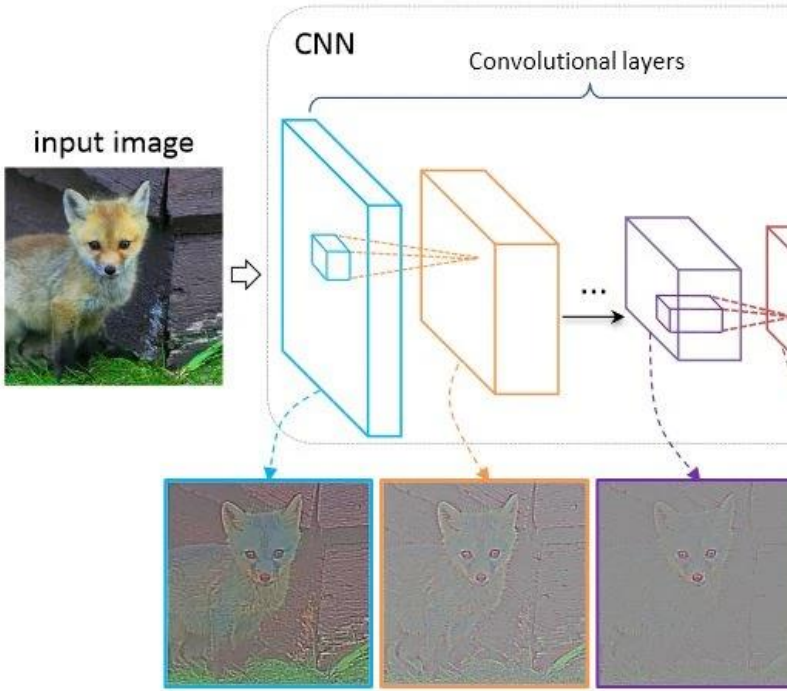


Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

VGG

Table 1: **ConvNet configurations** (shown in columns). The depth of the configurations increases from the left (A) to the right (E), as more layers are added (the added layers are shown in bold). The convolutional layer parameters are denoted as "conv⟨receptive field size⟩-⟨number of channels⟩". The ReLU activation function is not shown for brevity.

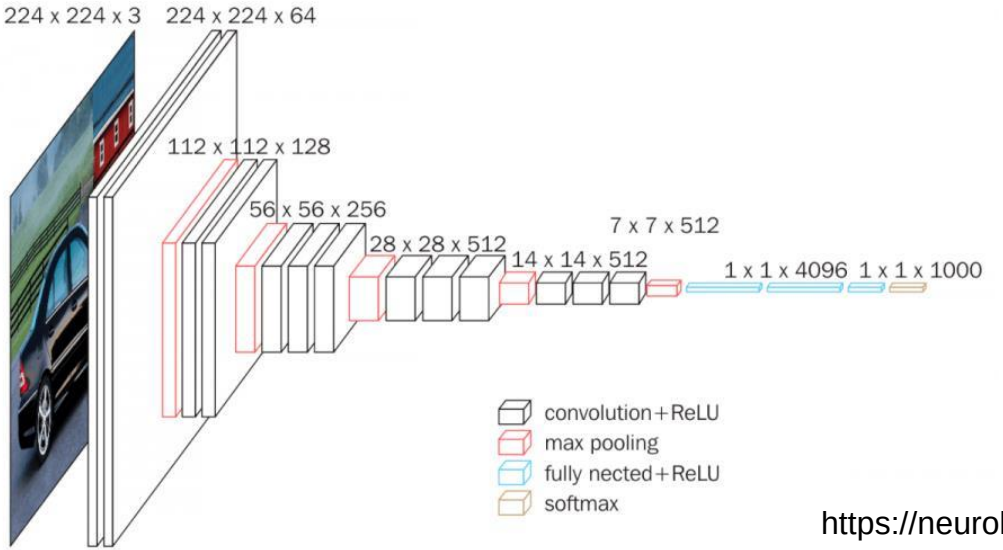
ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

VGG16

- VGG16 모델
  - VGG-16은 2014년 Oxford 대학의 VGG(Visual Geometry Group) 팀이 제안한 CNN 모델
  - ImageNet 대회(ILSVRC 2014)에서 우수한 성능을 기록
  - 이름의 16은 학습 가능한 레이어 수 → Convolution + Fully Connected 층 = 총 16개



https://neurohive.io/en/popular-networks/vgg16/

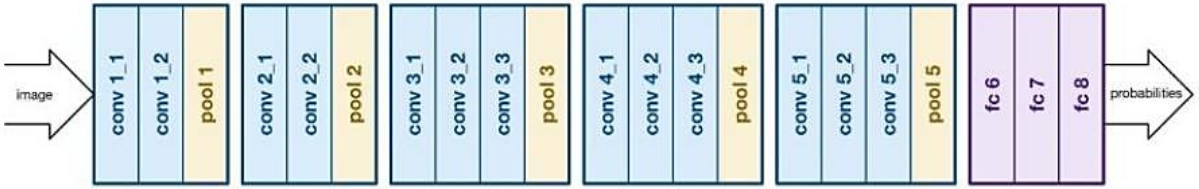
VGG16

- VGG16 모델
  - 모든 Convolution 층에서 3×3 필터만 사용
  - 작은 필터를 여러 번 쌓아 복잡한 특징을 단계적으로 학습
  - 구조가 단순하고 규칙적 → 이해와 구현이 쉬움
  - 입력 이미지 크기: 224 × 224 × 3

VGG16

- VGG16 모델

위 그림을 단순화시켜서 그려보면 아래와 같습니다.



너무 단순화시켰나요?

약간 추가정보를 넣어보면 다음과 같습니다.

3x3 필터를 통해
나온 64개 채널의
224 크기의 이미지

pooling layer 인데
vgg16에서는 max pooling 이라는
알고리즘을 사용합니다.

채널의 깊이가
깊어짐을 의미합니다.

박스로 되어있는 layer에서만
weight 가 학습됩니다.
vgg16이란 그런 박스 layer가
16개라는 의미입니다.

VGG19

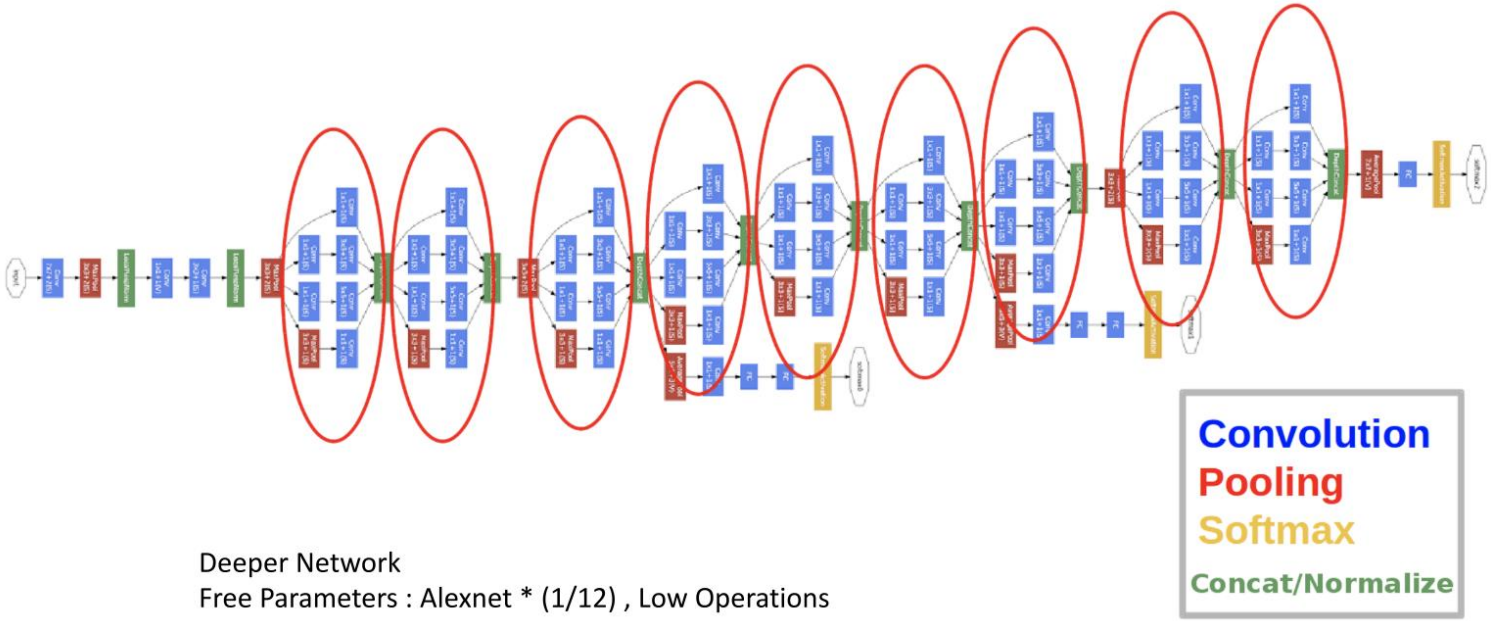
- VGG19 모델
  - VGG-16의 확장 버전
  - 총 19개의 학습 가능한 레이어
  - 더 깊은 네트워크로 표현력 향상이 목적
  - VGG-16과의 차이점
    - ✓ Convolution Layer가 더 많음
    - ✓ 필터 크기, 풀링 방식, FC 구조는 VGG-16과 동일
    - ✓ 차이는 오직 깊이(depth)

VGG19

Softmax
FC 1000
FC 4096
FC 4096
Pool
3x3 conv, 256
3x3 conv, 384
Pool
3x3 conv, 384
Pool
5x5 conv, 256
11x11 conv, 96
Input
AlexNet
Softmax
FC 1000
FC 4096
FC 4096
Pool
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
Pool
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
Pool
3x3 conv, 256
3x3 conv, 256
Pool
3x3 conv, 128
3x3 conv, 128
Pool
3x3 conv, 64
3x3 conv, 64
Input
VGG16
Softmax
FC 1000
FC 4096
FC 4096
Pool
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
Pool
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
Pool
3x3 conv, 256
3x3 conv, 256
Pool
3x3 conv, 128
3x3 conv, 128
Pool
3x3 conv, 64
3x3 conv, 64
Input
VGG19

GoogleNet

- GoogleNet
  - ➢ 신경망은 깊게 유지하여 성능을 올리면서도 파라미터 수를 줄여 효율성을 높일 수 있는 구조 제안

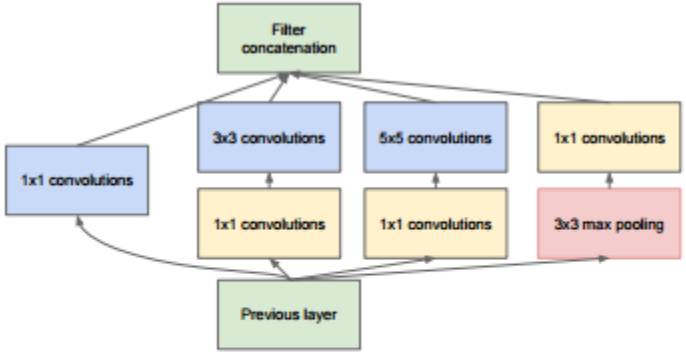


GoogleNet

- Naive Inception Module
  - 1*1 filter 활용: 이는 여러 feature map으로부터 특징을 추출하며 차원을 줄이는 효과를 가지기 때문에 연산량을 줄이는 데에도 효과적
  - 또한, 3*3, 5*5 filter를 병렬적으로 추가하여 입력으로부터 더 다양한 feature를 추출할 수 있음
  - 3*3 pooling도 병렬로 같이 수행하여 각 나온 결과를 concat하여 사용
  - 문제점
    - ✓ 5*5 filter가 많아질수록 연산량이 많아진다.
    - ✓ pooling layer의 특성상 이전 단계의 채널 사이즈를 유지한다. 다음 단계로 보낼 때 각 filter의 결과와 pooling의 결과를 concat하기 때문에 단계를 거칠수록 사이즈가 계속해서 커지는 것이 불가피하다.

GoogleNet

- 개선된 Inception Module (In GoogleNet)
  - 3*3, 5*5 filter를 적용하기 전 1*1 filter로 채널 수를 줄여줌
  - pooling layer를 통과한 후 이전 단계와 동일한 채널 사이즈를 1*1 filter를 통해 다시 줄여준 것을 볼 수 있음
  - 이렇게 사이즈를 줄이고 ReLU를 활성화 함수로 적용하여 비선형성을 추가



(b) Inception module with dimensionality reduction

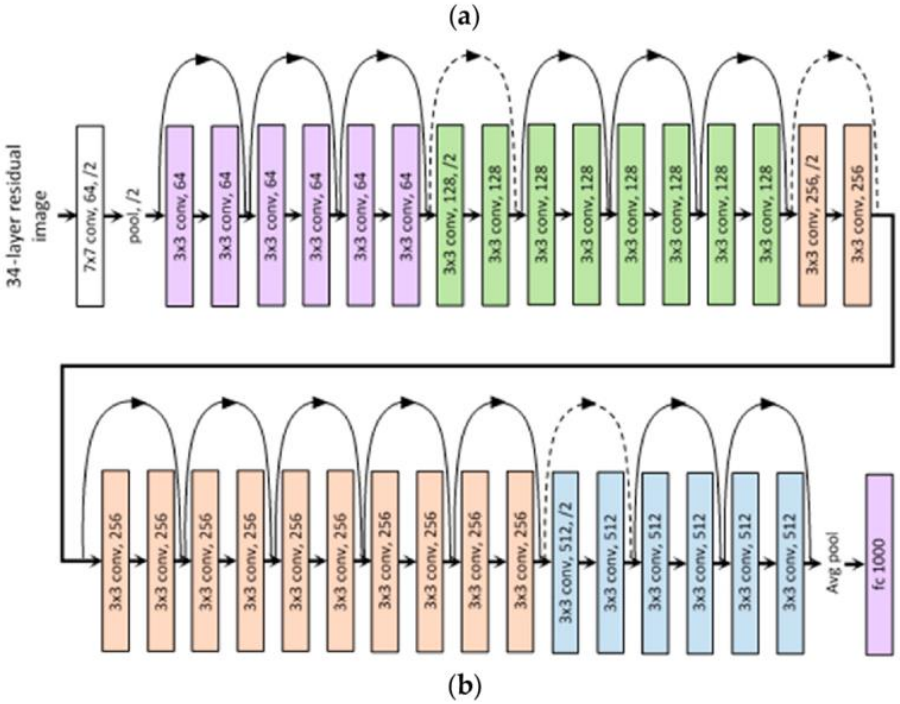
Figure 2: Inception module

ResNet

- ResNet
  - ➢ ILSVRC 2015 분류 문제에서 놀라운 성능 결과를 달성한 가장 강력한 심층 신경망 중 하나
  - ➢ ResNet 아키텍처에는 많은 변형이 존재
    - ✓ 즉, 동일한 개념이지만 레이어 수가 다름
    - ✓ ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-110, ResNet-152, ResNet-164, ResNet-1202 등

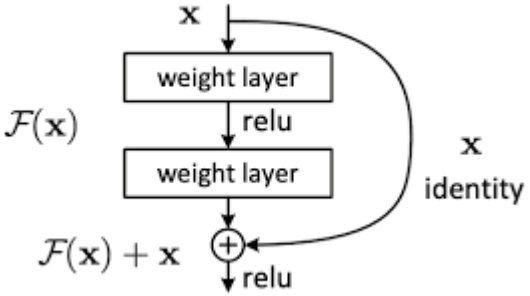
ResNet

- ResNet



ResNet

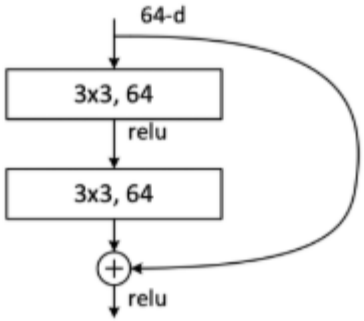
- Residual Learning
  - 기존 CNN은 입력 값(x)을 타깃 값(y)으로 매핑하는 함수 H(x)를 얻는 것이 목적
  - 그러나 ResNet은 F(x)+x를 최소화하는 것을 목적으로 함. x는 현 시점에서 변할 수 없는 값이므로 F(x)를 0에 가깝게 만드는 것이 목적이 됨
  - F(x)=H(x)-x 이므로 F(x)를 0에 가깝게 한다는 말은 H(x)-x를 0과 가깝게 하는 말과 동일한 의미
  - H(x)-x를 Residual 이라 함

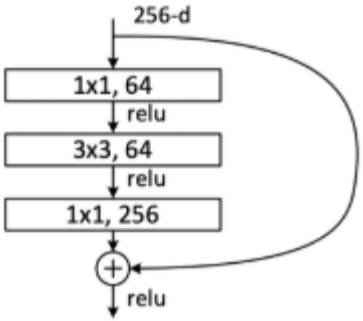


ResNet

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNet


64-d
3x3, 64
relu
3x3, 64
relu


256-d
1x1, 64
relu
3x3, 64
relu
1x1, 256
relu

```
def residual_block(x, filters, stride=1):

    shortcut = x

    # 첫 번째 Conv
    x = layers.Conv2D(filters, (3, 3),
                      strides=stride,
                      padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

 # 두 번째 Conv
    x = layers.Conv2D(filters, (3, 3), strides=1, padding='same')(x)
    x = layers.BatchNormalization()(x)
```

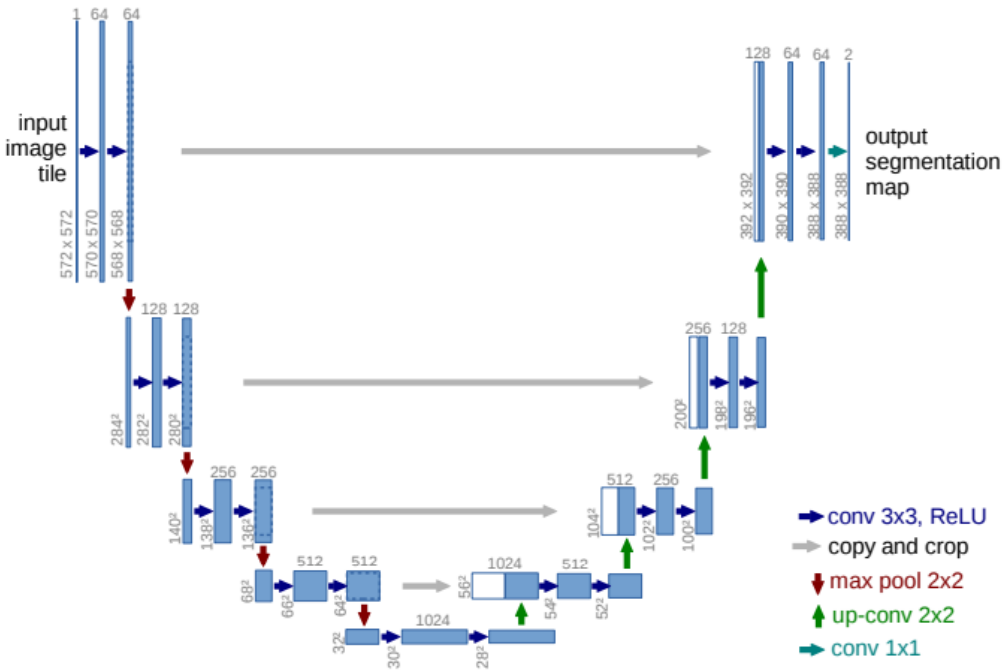
```
# shortcut 크기 맞추기
if stride != 1 or shortcut.shape[-1] != filters:
    shortcut = layers.Conv2D(filters, (1, 1), strides=stride, padding='same')(shortcut)
    shortcut = layers.BatchNormalization()(shortcut)

# Skip Connection
x = layers.Add()([x, shortcut])
x = layers.ReLU()(x)

return x
```

```
def build_resnet(input_shape=(32, 32, 3), num_classes=10):

    inputs = layers.Input(shape=input_shape)

    x = layers.Conv2D(64, (3, 3),
                      padding='same')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)

    # Block 1
    x = residual_block(x, 64)
    x = residual_block(x, 64)

    # Block 2
    x = residual_block(x, 128, stride=2)
    x = residual_block(x, 128)
```

```
# Block 3
x = residual_block(x, 256, stride=2)
x = residual_block(x, 256)

# Global Average Pooling
x = layers.GlobalAveragePooling2D()(x)

outputs = layers.Dense(num_classes, activation='softmax')(x)

model = models.Model(inputs, name="ResNet")

return model
```

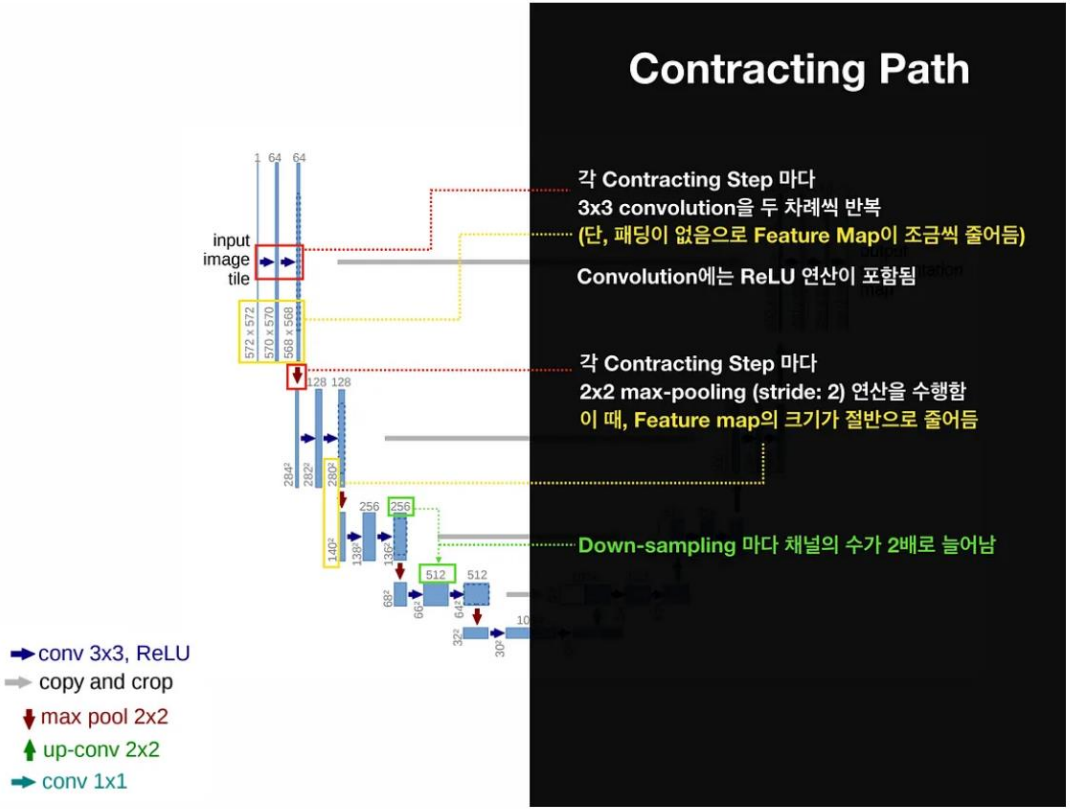
U-Net

- U-Net
  - ➢ U자 형태를 가지고 있는 모델
  - ➢ 크게 Contracting Path(축소 경로), Bottleneck(병목), Expansive Path(확장 경로)로 구분



U-Net

- U-Net
  - ➢ Contracting Path



U-Net

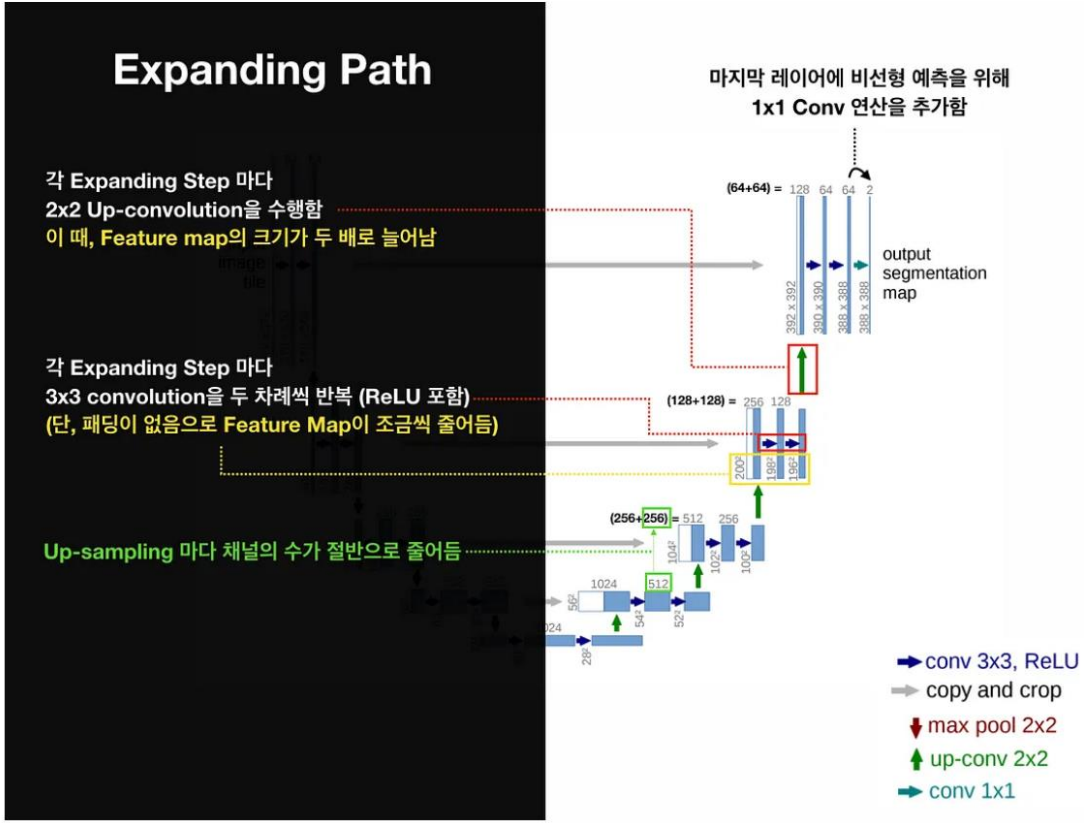
- U-Net
  - ➢ Contracting Path
    - ✓ 이 부분은 일반적인 CNN과 유사하며, 이미지의 컨텍스트를 포착하기 위해 사용됨
    - ✓ 여러개의 컨볼루션 레이어와 풀링레이어를 통해 이미지의 크기를 점차 축소 시켜나감
    - ✓ Contracting step마다 3x3 Convolution을 두번씩 반복하며, 해당 연산간 ReLU 연산이 포함
    - ✓ Down Sampling마다 채널의 수는 2배씩 증가하지만, 2x2 Max pooling으로 인해 feature map의 크기는 절반으로 줄어듦

U-Net

- U-Net
  - ➢ Bottleneck
    - ✓ Contracting path와 Expanding path를 연결하는 중심부분을 의미
    - ✓ 네트워크 이미지 상 중심에 위치(그림 상 가장 아래부분)하고 있으며, 입력 이미지의 가장 추상적이고 고수준의 표현을 학습하는 부분

U-Net

- U-Net
  - ➢ Expansive Path

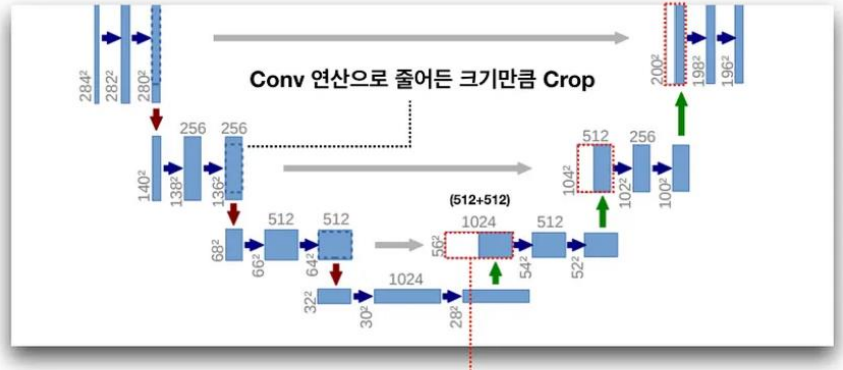


U-Net

- U-Net
  - ➢ Expansive Path
    - ✓ Bottleneck 다음에 위치하는 부분으로 이미지의 공간차원을 점차적으로 확장시켜나가며 세밀한 세그멘테이션을 가능하게 함
    - ✓ 각 Expanding step마다 2x2 Up-Convolution을 반복하며, 해당 연산간 ReLU 연산을 포함
    - ✓ 각 Up-sampling마다 채널의 수는 2배씩 감소하지만, 크기가 2x2 Up Convolution으로 인해 feature map의 크기가 2배씩 증가

U-Net

- U-Net
  - ➢ Skip Connection



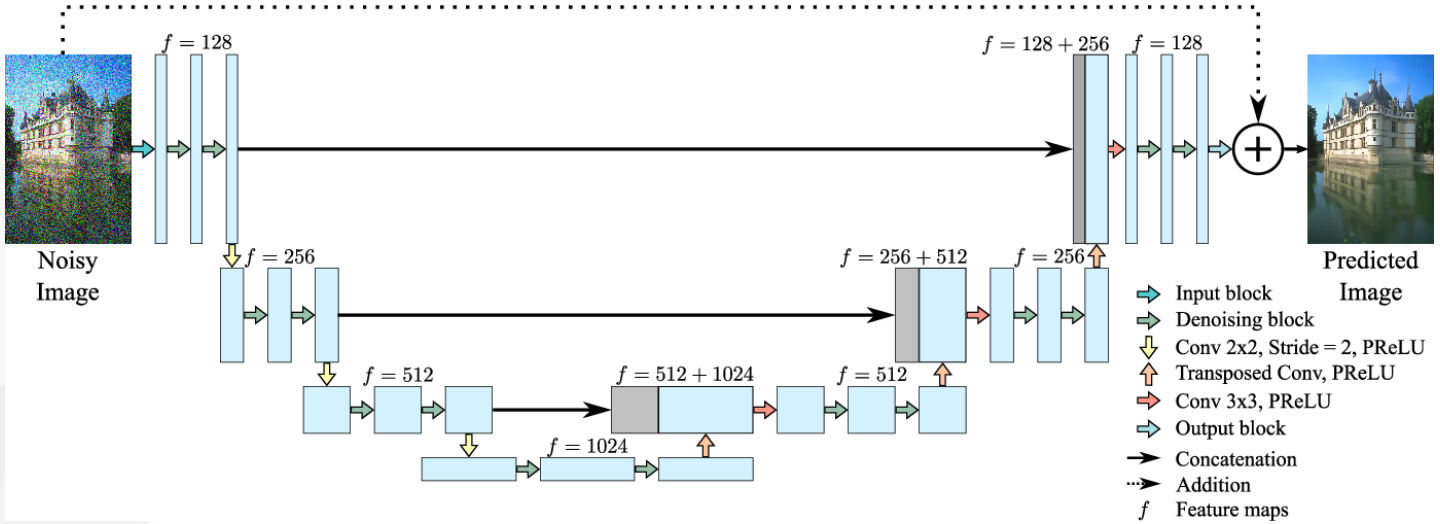
각 Expanding Step 마다
Up-conv 된 특징맵은 Contracting path의 Cropped된 특징맵과
Concatenation 함

conv 3x3, ReLU
copy and crop
max pool 2x2
up-conv 2x2
conv 1x1

Skip Architecture

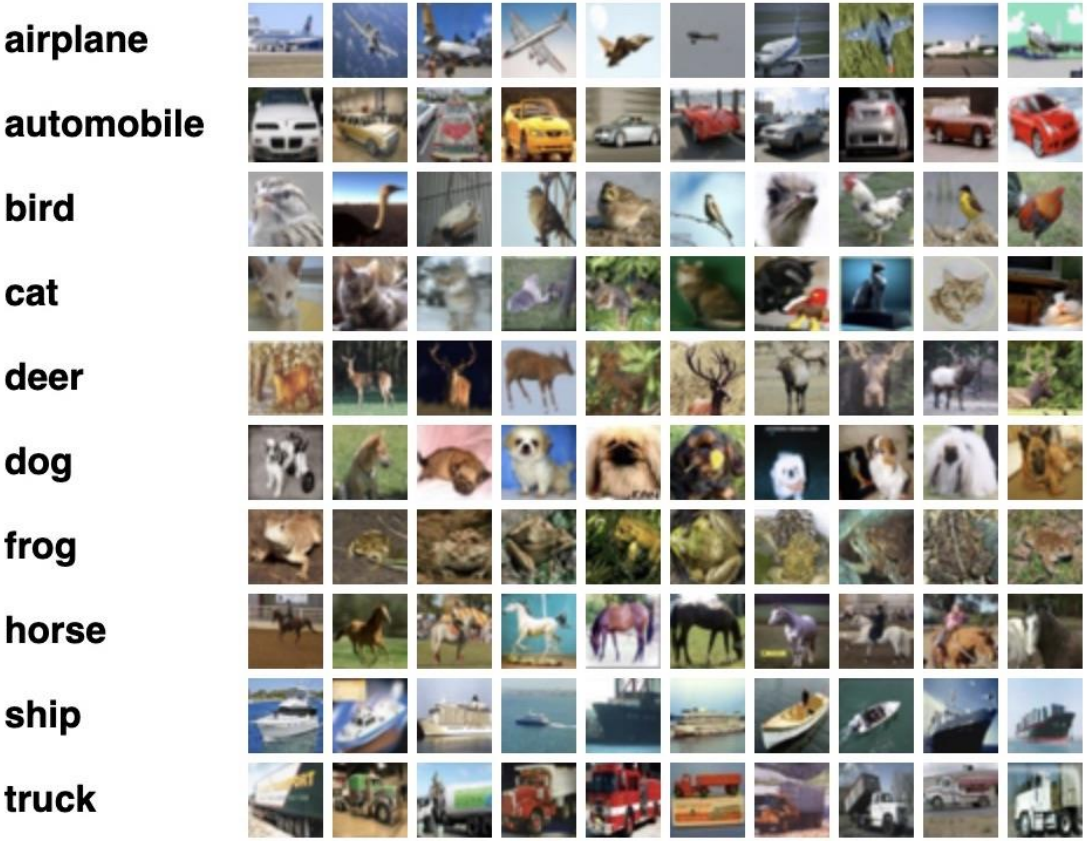
U-Net

- U-Net 변형
  - ➢ 입력과 출력의 크기를 갖게 설정
  - ➢ 복원 분야에서 많이 활용



과제

- CIFAR10 Dataset



과제

- CIFAR10 Dataset을 이용한 최고 정확도 달성
  - CIFAR10 Dataset을 이용해 최고 성능을 달성해 보자!
  - CIFAR10 Dataset : 32x32x3 (컬러 영상)
  - 본인만의 CNN 네트워크를 구성, 학습 방법 변경 등 (#6, #8, #9번만 변경 가능)
  - 소개했던(VGG, Resnet 등) 모델 혹은 하지 않았던 기존 모델 구조 검색을 통해 활용 가능
  - 주피터 노트북 파일과 h5 파일제출 (모델학습 부분과 정확도가 출력되어 있어야 함)

```
from tensorflow.keras.datasets import cifar10

(X_train, Y_train), (X_test, Y_test) = cifar10.load_data()
```

```
# 1. 라이브러리 import
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import models, layers
from tensorflow.keras.callbacks import EarlyStopping

# 2. 데이터 로드
(X_train, Y_train), (X_test, Y_test) = cifar10.load_data()

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

# 3. 데이터 정규화
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# 4. one-hot encoding
Y_train = to_categorical(Y_train, 10)
Y_test = to_categorical(Y_test, 10)
```

```
#5 Early Stopping, Checkpoint 설정
early_stop = EarlyStopping(
monitor='val_loss',
patience=5,
restore_best_weights=True
)

checkpoint = ModelCheckpoint(
filepath="best_model.h5",
monitor="val_loss",
save_best_only=True,
verbose=1
)
```

```
# 6. 모델 생성
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# 7. 모델 구조 확인
model.summary()
```

```
# 8. 모델 컴파일
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 9. 모델 학습
history = model.fit(
    X_train,
    Y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2,
    callbacks=[early_stop, checkpoint]
)

# 10. 모델 평가
test_loss, test_acc = model.evaluate(X_test, Y_test)

print("Test accuracy:", test_acc)
```

실습

- 2012년 ILSVRC 대회에서 우승하여 CNN 부흥에 큰 역할을 한 모델은 무엇인가?
- AlexNet에서 사용한 특별한 Pooling 방식의 이름은 무엇인가?
- AlexNet에서 Fully Connected Layer의 노드 수는 각각 몇 개인가?
- VGG16에서 모든 Convolution Layer에 공통적으로 사용된 필터 크기는 무엇인가?
- GoogleNet에서 여러 크기의 필터를 병렬로 사용하는 구조를 무엇이라고 하는가?
- GoogleNet에서 연산량 감소를 위해 사용하는 1×1 필터의 주요 목적은 무엇인가?
- GoogleNet에서 다양한 특징을 동시에 추출하기 위해 병렬로 사용하는 필터 크기 두 가지를 쓰시오.

실습

- U-Net에서 이미지 크기를 점점 줄여 특징을 추출하는 경로를 무엇이라고 하는가?
- U-Net에서 가장 중심에 위치하여 가장 추상적인 특징을 학습하는 부분은 무엇인가?
- CIFAR-10 데이터셋 이미지 크기는 무엇인가?
- 여러 연산 결과를 하나로 합치는 연산을 무엇이라고 하는가?
- ResNet에서 동일한 입력을 더하는 연산을 수행하는 블록을 무엇이라고 하는가?
- ResNet-50에서 숫자 50은 무엇을 의미하는가?
- U-Net에서 Contracting Path와 Expansive Path를 연결하는 구조를 무엇이라고 하는가?
- VGG16의 입력 이미지 크기는 무엇인가?

참고 문헌

- 케라스 창시자에게 배우는 딥러닝, 프랑소와 숄레 지음, 박해선 옮김, 길벗
- 권건우, 야사와 만화로 배우는 인공지능, 루나파인북스
- https://wikidocs.net/164800
- https://m.blog.naver.com/baek2sm/222722120152
- https://wjunsea.tistory.com/99
- https://jaylala.tistory.com/entry/%EB%94%A5%EB%9F%AC%EB%8B%9D-with-%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0U-Net%EC%9D%B4%EB%9E%80-U-Net-Convolutional-Networks-for-Biomedical-Image-Segmentation